

Guia Essencial: Principais Componentes do C# com Exemplos Práticos

1. Variáveis e Tipos de Dados: A Base do Código

Variáveis armazenam informações que seu programa pode usar. Em C#, você deve definir o tipo da variável.

Exemplo Prático:

```
int idade = 25;  
string nome = "João";  
bool ativo = true;
```

```
Console.WriteLine($"Nome: {nome}, Idade: {idade}, Ativo: {ativo}");
```

- **Dica:** Use tipos adequados para economizar memória e evitar erros.
-

2. Estruturas Condicionais: Decisões Inteligentes no Código

As estruturas condicionais permitem que o programa tome decisões com base em condições específicas.

Exemplo Prático:

```
int nota = 85;  
if (nota >= 60)  
{  
    Console.WriteLine("Aprovado!");  
}  
else  
{  
    Console.WriteLine("Reprovado!");  
}
```

- **Dica:** Use switch para múltiplas condições simples.
-

3. Loops: Repetição Eficiente de Tarefas

Loops permitem repetir blocos de código sem duplicação.

Exemplo Prático:

```
for (int i = 1; i <= 5; i++)  
{  
    Console.WriteLine($"Número: {i}");  
}
```

- **Dica:** Use foreach para percorrer listas.
-

4. Classes e Objetos: Organização do Código

Classes são modelos que definem propriedades e comportamentos. Objetos são instâncias dessas classes.

Exemplo Prático:

```
class Carro
{
    public string Modelo { get; set; }
    public void Ligar() => Console.WriteLine("Carro ligado");
}
```

```
Carro meuCarro = new Carro { Modelo = "Fusca" };
meuCarro.Ligar();
```

- **Dica:** Encapsule dados sensíveis usando private.
-

5. Métodos: Reutilize e Organize Funcionalidades

Métodos agrupam lógica reutilizável.

Exemplo Prático:

```
void MostrarMensagem(string mensagem)
{
    Console.WriteLine(mensagem);
}
```

```
MostrarMensagem("Olá, mundo!");
```

- **Dica:** Prefira métodos pequenos e específicos.
-

6. Exceções: Lidando com Erros de Forma Elegante

Exceções ajudam a tratar erros sem quebrar o programa.

Exemplo Prático:

```
try
{
    int resultado = 10 / 0;
}
catch (DivideByZeroException ex)
{
    Console.WriteLine("Erro: divisão por zero não permitida.");
}
```

```
}
```

- **Dica:** Sempre use try-catch para operações críticas.
-

7. Listas: Manipule Coleções de Dados

Listas permitem armazenar múltiplos elementos de forma dinâmica.

Exemplo Prático:

```
List<string> frutas = new List<string> { "Maçã", "Banana", "Uva" };
foreach (var fruta in frutas)
{
    Console.WriteLine(fruta);
}
```

- **Dica:** Use LINQ para consultas mais avançadas.
-

8. LINQ: Consultas Poderosas em Dados

LINQ facilita consultas em coleções e bancos de dados.

Exemplo Prático:

```
List<int> numeros = new List<int> { 1, 2, 3, 4, 5 };
var pares = numeros.Where(n => n % 2 == 0);

foreach (var num in pares)
{
    Console.WriteLine(num);
}
```

- **Dica:** LINQ é excelente para manipulação rápida de dados.
-